

Agentic Adversarial Debiasing: A Dynamic Multi-Agent Framework for Fairness-Aware Machine Learning

Wassim Fathallah¹ and Nahla Ben Amor¹

Institut Supérieur de Gestion de Tunis, Université de Tunis, Tunisia
LARODEC Laboratory
wassimfathallah10@gmail.com, nahla.benamor@gmx.fr

Abstract. Adversarial debiasing methods jointly optimize accuracy and fairness by framing the problem as a minimax game between a classifier and an adversary. Two structural limitations, however, limit their practical deployment: the fairness penalty weight α must be fixed before training with no adaptive mechanism, and sensitive attributes must be declared manually for every new dataset. We propose AADA (Agentic Adversarial Debiasing Algorithm), a three-agent framework that addresses both limitations simultaneously. A feedback-driven dynamic lambda adjustment mechanism replaces the fixed α with a per-attribute vector updated at each iteration, stabilized by momentum and a running-maximum guard. An LLM-based orchestrator identifies sensitive attributes automatically, enabling application to unseen datasets without manual configuration. AADA handles multiple sensitive attributes simultaneously, experiments on seven datasets from the Fair Fairness Benchmark (FFB) and HIMS-Tunisia show that AADA reaches or approaches the fairness target (P-rule ≥ 80) while replacing a multi-value penalty sweep with a single adaptive run.

Keywords: Algorithmic fairness · Adversarial debiasing · AI agents · Bias mitigation · Machine learning

1 Introduction

Machine learning systems are increasingly deployed in high-stakes domains such as hiring, credit scoring, and criminal justice, where their decisions directly affect people’s lives. A model trained to minimize a task loss on historical data may reproduce or amplify inequalities present in the data. Angwin et al. [1] revealed that COMPAS, a recidivism prediction tool widely used by American courts, assigned significantly higher risk scores to Black defendants than to White defendants with comparable criminal histories. That same year, Bolukbasi et al. [3] showed that word embeddings had internalized gender stereotypes, and Amazon shut down an internal recruiting tool after discovering it systematically penalized resumes from women [6]. Buolamwini and Gebru [4] further demonstrated that

commercial face recognition systems produced error rates up to 34 percentage points higher for darker-skinned women than for lighter-skinned men.

These cases motivate the field fairness-aware machine learning [2]. Among debiasing approaches, adversarial methods [35, 24] are particularly attractive: they reduce sensitive information encoded in predictions while jointly optimizing the predictive task. Two practical limitations, however, block their deployment.

First, the fairness penalty weight α is a fixed scalar that must be tuned by hand before training begins, with no mechanism to respond to the model’s evolving fairness state. Han et al. [14] confirm this empirically across 45,079 experiments: the results of adversarial debiasing are scattered across the accuracy–fairness space with no consistent trade-off pattern, making the method hard to control in practice. Second, the sensitive attributes must be identified and declared manually for every new dataset, requiring domain expertise and blocking application to unseen data.

We propose the **Agentic Adversarial Debiasing Algorithm (AADA)**, a framework organized around three cooperating agents in the classical sense [31]: an orchestrator that perceives the dataset schema and training signals, acts by identifying sensitive attributes automatically and controlling the adversarial training loop, and improves future runs through memory, a classifier that balances task accuracy against the fairness penalty, and an adversary whose success or failure drives the fairness correction. The central contribution is a feedback-driven dynamic lambda adjustment mechanism that replaces the fixed scalar α with a per-attribute vector $\lambda^{(t)}$ updated at each iteration in response to the measured fairness state, stabilized by a momentum term and a running-maximum guard. Crucially, AADA handles all sensitive attributes simultaneously in a single training run, whereas the considered benchmark baselines are evaluated through independent per-attribute runs.

We make three contributions:

1. **AADA**, an agentic adversarial debiasing framework in which a meta-level orchestrator identifies sensitive attributes, configures the training pipeline, monitors fairness, reuses memory from previous runs, and adaptively updates per-attribute penalties in a single multi-attribute training run.
2. **Automatic sensitive attribute identification** via a large language model sub-component of the orchestrator, eliminating manual configuration when applying the pipeline to a new tabular dataset.
3. **A dynamic fairness-control mechanism** based on per-attribute adaptive penalties, momentum smoothing, and a running-maximum guard for single-run multi-attribute debiasing.

The remainder of this paper is organized as follows. Section 2 reviews related work on fairness metrics and in-processing debiasing techniques. Section 3 presents the AADA framework. Section 4 reports experimental results. Section 5 concludes and discusses limitations.

2 Fairness Metrics and In-Processing Debiasing

Fairness in machine learning is not a single concept but a family of notions rooted in legal, ethical, and statistical reasoning [2, 30]. Two legal forms of discrimination are commonly distinguished: *disparate treatment*, where decisions explicitly depend on a sensitive attribute, and *disparate impact*, where apparently neutral decisions disproportionately affect protected groups [10]. At the metric level, widely used criteria include demographic parity [25], equalized odds [15], equal opportunity, and individual fairness [9]. Since several fairness criteria are known to be incompatible in general settings [5, 21], the choice of metric depends on the application context.

Bias mitigation methods are commonly grouped into three categories according to the stage at which they intervene in the machine learning pipeline [25, 27]. *Pre-processing* methods modify the training data before learning [18, 34]. They are often model-agnostic, but the learned model may still introduce or amplify bias during training. *Post-processing* methods adjust predictions after training [15]. They are useful when the model cannot be retrained, but they act only on the final outputs. *In-processing* methods integrate fairness directly into the learning objective and therefore control the accuracy–fairness trade-off during optimization. Because our goal is to adapt fairness constraints during training, we focus on the in-processing family.

Within in-processing approaches, Empirical Risk Minimization (ERM) [11] serves as an unconstrained baseline with no fairness intervention. Prejudice Remover (PR) [19] introduces a fairness regularization term into the loss function. Hilbert-Schmidt Independence Criterion regularization (HSIC) [23] reduces the statistical dependence between predictions and sensitive attributes through a kernel independence criterion.

The closest methods to our work are adversarial approaches. Adversarial Debiasing (AdvDebias) [35] formulates fairness as a minimax game between a classifier and an adversary attempting to recover sensitive attributes from model predictions. Learning Adversarially Fair and Transferable Representations (LAFTR) [24] extends this principle to representation learning through an encoder–decoder architecture. These approaches are particularly relevant because they do not only penalize direct dependence on sensitive attributes: they also aim to reduce sensitive information recoverable through proxies in predictions or learned representations.

However, standard adversarial debiasing remains difficult to deploy in practice. First, the sensitive attributes must be specified before training, which assumes that the user already knows which attributes should be protected in each dataset. Second, the adversarial penalty is usually fixed before training and selected through hyperparameter tuning. Han et al. [14] show that adversarial debiasing can produce highly variable accuracy–fairness trade-offs across datasets and penalty values, which makes manual tuning costly and unstable.

AADA builds on the adversarial foundation of AdvDebias and LAFTR, but moves fairness control to a meta-level orchestrator. The orchestrator identifies candidate sensitive attributes using a locally served LLM, monitors the fairness

state during training, updates a per-attribute penalty vector, and reuses memory from previous runs. This enables single-run multi-attribute debiasing without manually fixing a single fairness penalty or manually declaring all sensitive attributes in advance.

3 Agentic Adversarial Debiasing

3.1 Problem Formulation

Let $\mathcal{D} = \{(\mathbf{x}_n, y_n, \mathbf{z}_n)\}_{n=1}^N$ be a dataset where $\mathbf{x}_n \in \mathbb{R}^d$ is the feature vector of instance n , $y_n \in \{0, 1\}$ the binary target label, and $\mathbf{z}_n = (z_{n,1}, \dots, z_{n,n_s})$ the observed values of the n_s binary sensitive attributes, with $z_{n,i} \in \{0, 1\}$ for each $i \in \{1, \dots, n_s\}$. Let $\boldsymbol{\theta}_f$ and $\boldsymbol{\theta}_r$ denote the trainable parameters of the classifier and adversary respectively. The classifier $f_{\boldsymbol{\theta}_f} : \mathbb{R}^d \rightarrow (0, 1)$ produces a probabilistic prediction $\hat{y}_n = f_{\boldsymbol{\theta}_f}(\mathbf{x}_n)$; $\tilde{y}_n = \mathbb{I}[\hat{y}_n \geq 0.5] \in \{0, 1\}$ denotes the corresponding binary prediction. The adversary $r_{\boldsymbol{\theta}_r} : (0, 1) \rightarrow \mathbb{R}^{n_s}$ receives only $\hat{y}_n$ and outputs one logit per sensitive attribute $\hat{z}_{ni}$. The held-out evaluation split is denoted $\mathcal{D}_{\text{eval}}$.

Following Zhang et al. [35] and the minimax principle extended to deep learning by Goodfellow et al. [12], we frame the problem as a **zero-sum minimax game** between the classifier and the adversary. Our orchestrator controls the intensity of this competition through $\boldsymbol{\lambda}^{(t)}$ (Section 3.6). Formally:

$$\min_{\boldsymbol{\theta}_f} \max_{\boldsymbol{\theta}_r} \left[\mathcal{L}_f(\boldsymbol{\theta}_f) - \sum_{i=1}^{n_s} \lambda_i \mathcal{L}_{r,i}(\boldsymbol{\theta}_r; \boldsymbol{\theta}_f) \right], \quad (1)$$

where $\lambda_i \geq 0$ controls the fairness pressure for attribute i . In AADA, this penalty is later made iteration-dependent and written $\lambda_i^{(t)}$.

3.2 Three-Agent Architecture

We organize AADA as an agentic control framework. In this work, “agentic” refers to an orchestrator that perceives the dataset schema and the evolving accuracy–fairness state, acts by configuring the pipeline and updating $\boldsymbol{\lambda}^{(t)}$, and improves future runs by storing and retrieving past experience [31]. The classifier and the adversary remain reactive optimization agents within the minimax game.

- The **orchestrator** is the central learning agent of AADA. It observes the dataset schema and a small sample of rows, uses a locally served LLM to infer the prediction target and candidate sensitive attributes, and configures the training pipeline without exposing the data to an external service. During training, it monitors the accuracy–fairness state after each adversarial iteration and acts by updating the per-attribute penalty vector $\boldsymbol{\lambda}^{(t)}$, checking the stopping condition, and selecting the returned model. Across runs, it stores a memory containing the dataset fingerprint, detected sensitive attributes, accuracy–fairness trajectory, and best-performing lambda vector.

When a new dataset is processed, the orchestrator retrieves the most similar previous run and uses its best lambda vector to initialize $\lambda^{(0)}$, if no similar run is available, $\lambda^{(0)}$ is initialized from a default value.

- The **classifier** is a utility-based agent $f_{\theta_f} : \mathbb{R}^d \rightarrow (0, 1)$, optimizing a composite objective balancing task accuracy against the adversarial penalty, its architecture is dataset-dependent and detailed in Section 4.
- The **adversary** is a goal-based agent $r_{\theta_r} : (0, 1) \rightarrow \mathbb{R}^{n_s}$, receiving only $\hat{y}_n$ and outputting one logit per sensitive attribute. This restriction is intentional: weak adversary performance suggests that less sensitive information is recoverable from the classifier output.

Figure 1 gives an overview of the three-agent architecture.

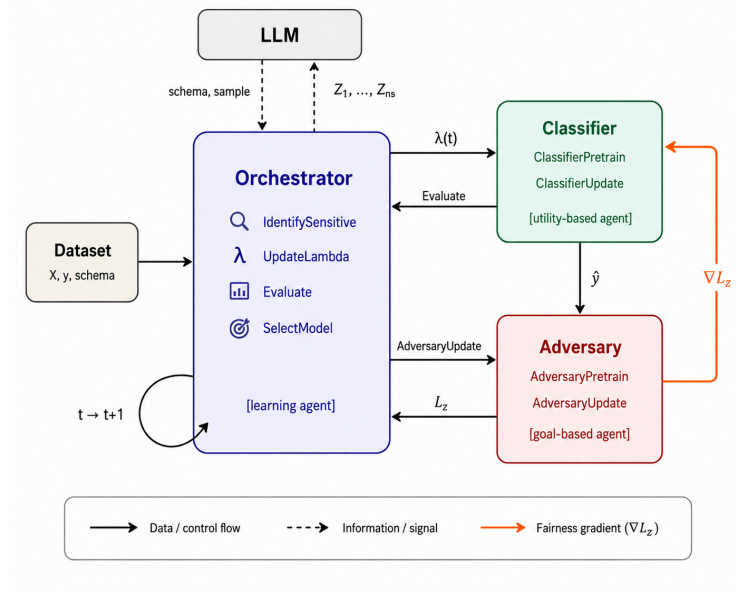


Fig. 1. Overview of the AADA three-agent architecture.

3.3 Sensitive Attribute Identification

At initialization, the orchestrator calls `IDENTIFYSENSITIVE` (Algorithm 1), which receives the dataset schema and a small sample of rows and uses a large language model to return the prediction target Y , the sensitive attributes $Z_1, \dots, Z_{n_s}$, and the columns to drop before training. This step is performed once and does not interact with the adversarial loop. The LLM is required to return a structured tuple $(Y, \mathcal{Z}, \mathcal{C})$, where Y is the prediction target, $\mathcal{Z}$ is the set of candidate sensitive attributes, and $\mathcal{C}$ is the set of columns to remove before training.

Algorithm 1 IDENTIFYSENSITIVE: Sensitive Attribute Identification**Require:** dataset schema, sample rows

- 1: Build prompt from schema and sample
- 2: Query large language model, collect response
- 3: Parse response into $Y, Z_1, \dots, Z_{n_s}, \text{cols_drop}$
- 4: **return** $Y, Z_1, \dots, Z_{n_s}, \text{cols_drop}$

3.4 Adversarial Training Protocol

Pre-training. We warm-start both agents over N_{pre} epochs in a non-adversarial phase: at each batch, the classifier minimizes $\mathcal{L}_f$, then the adversary minimizes $\mathcal{L}_r$ on the detached predictions, with θ_f held constant during the adversary step. The classifier loss over mini-batch B is the binary cross-entropy:

$$\mathcal{L}_f(\theta_f; B) = -\frac{1}{|B|} \sum_{n \in B} [y_n \log \hat{y}_n + (1 - y_n) \log(1 - \hat{y}_n)]. \quad (2)$$

The adversary loss for attribute i is the binary cross-entropy with logits:

$$\mathcal{L}_{r,i}(\theta_r; \theta_f, B) = \frac{1}{|B|} \sum_{n \in B} \text{BCELogits}(\hat{z}_{ni}, z_{ni}), \quad (3)$$

where, for a logit s and binary label z , $\text{BCELogits}(s, z) = \max(s, 0) - sz + \log(1 + e^{-|s|})$, and $\mathcal{L}_r = \sum_{i=1}^{n_s} \mathcal{L}_{r,i}$.

Per-batch adversarial updates. At Step 1, the classifier is frozen (**no_grad**) and the adversary minimizes $\mathcal{L}_r$ on detached predictions by a gradient step of the form below, where $\eta_r > 0$ is the adversary learning rate:

$$\theta_r \leftarrow \theta_r - \eta_r \nabla_{\theta_r} \mathcal{L}_r(\theta_r; \theta_f, B). \quad (4)$$

At Step 2, the adversary is frozen and the classifier minimizes the composite loss, adapted from Zhang et al. [35]:

$$\mathcal{L}_{\text{total}}(\theta_f; B) = \mathcal{L}_f(\theta_f; B) - \sum_{i=1}^{n_s} \lambda_i^{(t)} \mathcal{L}_{r,i}(\theta_r; \theta_f, B). \quad (5)$$

The gradient decomposes as:

$$\nabla_{\theta_f} \mathcal{L}_{\text{total}} = \nabla_{\theta_f} \mathcal{L}_f - \sum_{i=1}^{n_s} \lambda_i^{(t)} \nabla_{\theta_f} \mathcal{L}_{r,i}. \quad (6)$$

$\hat{y}_n$ is passed without detaching, so the fairness signal flows back via the chain rule:

$$\nabla_{\theta_f} \mathcal{L}_{r,i} = \frac{\partial \mathcal{L}_{r,i}}{\partial \hat{\mathbf{z}}} \cdot \frac{\partial \hat{\mathbf{z}}}{\partial \hat{y}_n} \cdot \frac{\partial \hat{y}_n}{\partial \theta_f}. \quad (7)$$

Equations (6) and (7) show that the fairness signal reaches θ_f only through $\hat{y}_n$: the classifier is penalized solely for the demographic information encoded in its output.

3.5 Evaluation Metrics

At the end of each iteration, the orchestrator evaluates the classifier on $\mathcal{D}_{\text{eval}}$:

$$\text{acc}^{(t)} = \frac{1}{|\mathcal{D}_{\text{eval}}|} \sum_{n \in \mathcal{D}_{\text{eval}}} \mathbb{I}[\tilde{y}_n^{(t)} = y_n]. \quad (8)$$

The positive prediction rate for group $a \in \{0, 1\}$ of attribute Z_i is [25]:

$$\pi_{i,a}^{(t)} = \frac{\sum_n \mathbb{I}[\tilde{y}_n^{(t)} = 1 \wedge z_{ni} = a]}{\sum_n \mathbb{I}[z_{ni} = a] + \varepsilon}, \quad (9)$$

where $\varepsilon > 0$ prevents division by zero. The P-rule, a symmetric reformulation of the $p\%$ -rule of Zafar et al. [33], is:

$$\text{P-rule}_i^{(t)} = 100 \times \frac{\min(\pi_{i,0}^{(t)}, \pi_{i,1}^{(t)}) + \varepsilon}{\max(\pi_{i,0}^{(t)}, \pi_{i,1}^{(t)}) + \varepsilon}. \quad (10)$$

A value of 100 indicates perfect demographic parity, 80 corresponds to the EEOC four-fifths threshold [10].

3.6 Dynamic Lambda Adjustment

Standard adversarial debiasing fixes α before training [14]. We propose to replace this fixed scalar with a per-attribute penalty vector $\boldsymbol{\lambda}^{(t)}$ that adapts at each iteration in response to the measured P-rule. For each sensitive attribute i , we define the normalized fairness gap as:

$$\delta_i^{(t)} = \frac{\tau_{\text{fair}} - \text{P-rule}_i^{(t)}}{100}. \quad (11)$$

We smooth the increment using momentum, where $\beta \in [0, 1]$ is the momentum coefficient and $\eta_\lambda > 0$ the fairness learning rate:

$$m_i^{(t)} = \beta m_i^{(t-1)} + (1 - \beta) \eta_\lambda \delta_i^{(t)}. \quad (12)$$

We update and clamp λ_i to $[0, \lambda_{\text{max}}]$, where $\lambda_{\text{max}} > 0$ is an upper bound on the penalty:

$$\lambda_i^{(t+1)} = \text{clip}(\lambda_i^{(t)} + m_i^{(t)}, 0, \lambda_{\text{max}}). \quad (13)$$

To prevent premature relaxation, we introduce a running-maximum guard: when $\text{P-rule}_i^{(t)} < \tau_{\text{fair}}$, λ_i cannot fall below its historical maximum:

$$\lambda_i^{(t+1)} \leftarrow \max(\lambda_i^{(t+1)}, \Lambda_i^{(t)}), \quad (14)$$

where $\Lambda_i^{(t+1)} \leftarrow \max(\Lambda_i^{(t)}, \lambda_i^{(t+1)})$ is updated unconditionally at every iteration.

The following property states the role of the running-maximum guard: while an attribute remains below the fairness target, the fairness pressure assigned to that attribute cannot decrease.

Proposition 1. *The running-maximum guard (Eq. (14)) ensures that $\lambda_i^{(t)}$ is non-decreasing as long as $\text{P-rule}_i^{(t)} < \tau_{\text{fair}}$.*

Proof. We show by induction that $\Lambda_i^{(t)} = \max_{s \leq t} \lambda_i^{(s)}$. The base case $\Lambda_i^{(0)} = \lambda_i^{(0)}$ holds by initialization. If $\Lambda_i^{(t)} = \max_{s \leq t} \lambda_i^{(s)}$, then $\Lambda_i^{(t+1)} = \max(\Lambda_i^{(t)}, \lambda_i^{(t+1)}) = \max_{s \leq t+1} \lambda_i^{(s)}$. Hence, when the guard is active, $\lambda_i^{(t+1)} \geq \Lambda_i^{(t)} \geq \lambda_i^{(t)}$.

Algorithm 2 summarizes the complete procedure.

Algorithm 2 UPDATELAMBDA: Dynamic Fairness Penalty Adjustment

Require: $\{\text{P-rule}_i^{(t)}\}, \boldsymbol{\lambda}^{(t)}, \mathbf{m}^{(t-1)}, \boldsymbol{\Lambda}^{(t)}$

- 1: **for** $i = 1$ **to** n_s **do**
- 2: Compute $\delta_i^{(t)}$ using Eq. (11)
- 3: Compute $m_i^{(t)}$ using Eq. (12)
- 4: Compute $\lambda_i^{(t+1)}$ using Eq. (13)
- 5: **if** $\text{P-rule}_i^{(t)} < \tau_{\text{fair}}$ **then**
- 6: Apply running-max guard Eq. (14)
- 7: **end if**
- 8: $\Lambda_i^{(t+1)} \leftarrow \max(\Lambda_i^{(t)}, \lambda_i^{(t+1)})$
- 9: **end for**
- 10: **return** $\boldsymbol{\lambda}^{(t+1)}, \mathbf{m}^{(t)}, \boldsymbol{\Lambda}^{(t+1)}$

3.7 Stopping Condition and Model Selection

We terminate the adversarial loop as soon as the fairness target is met on all attributes simultaneously:

$$\min_i \text{P-rule}_i^{(t)} \geq \tau_{\text{fair}}. \quad (15)$$

When Eq. (15) is first satisfied, the current model is returned as **optimal**. If the target is never satisfied, we select the model maximizing $\min_i \text{P-rule}_i^{(t)}$ (**best_trade_off**), if several models tie, we keep the most accurate one.

3.8 AADA Algorithm

Algorithm 3 states the complete pipeline, orchestrating IDENTIFYSENSITIVE, pre-training, per-batch adversarial updates, UPDATELAMBDA, and SELECTMODEL.

Algorithm 3 AADA: Agentic Adversarial Debiasing Algorithm

Require: $\mathcal{D}$, N_{pre} , E , b , τ_{fair} , η_f , η_r , η_λ , β , λ_{max} , $\boldsymbol{\lambda}^{\text{init}}$
 1: $Y, Z_1, \dots, Z_{n_s} \leftarrow \text{IDENTIFYSENSITIVE}(\mathcal{D}.\text{schema}, \mathcal{D}.\text{sample})$
 2: Preprocess $\mathcal{D}$ into $\mathcal{D}_{\text{train}}$, $\mathcal{D}_{\text{eval}}$
 3: Initialize $f_{\boldsymbol{\theta}_f}$, $r_{\boldsymbol{\theta}_r}$ with random weights
 4: $\lambda_i^{(0)} \leftarrow \lambda_i^{\text{init}}$, $m_i^{(0)} \leftarrow 0$, $\Lambda_i^{(0)} \leftarrow \lambda_i^{(0)} \quad \forall i$
 5: Pretrain $f_{\boldsymbol{\theta}_f}$ and $r_{\boldsymbol{\theta}_r}$ for N_{pre} epochs $\triangleright$ Eqs. (2), (3)
 6: **repeat**
 7: **for** $e = 1$ **to** E **do**
 8: **for** each mini-batch B of size b in $\mathcal{D}_{\text{train}}$ **do**
 9: Update $\boldsymbol{\theta}_r$ on detached $\hat{y}_n$ $\triangleright$ Eq. (4)
 10: Update $\boldsymbol{\theta}_f$ via $\mathcal{L}_{\text{total}}$ $\triangleright$ Eqs. (5), (7)
 11: **end for**
 12: **end for**
 13: Compute $\text{acc}^{(t)}$, $\{\text{P-rule}_i^{(t)}\}$ on $\mathcal{D}_{\text{eval}}$ $\triangleright$ Eqs. (8), (10)
 14: **if** Eq. (15) is satisfied **then**
 15: **return** $f_{\boldsymbol{\theta}_f}$, $\text{acc}^{(t)}$, $\{\text{P-rule}_i^{(t)}\}$, **optimal**
 16: **end if**
 17: $\boldsymbol{\lambda}^{(t+1)}, \mathbf{m}^{(t)}, \boldsymbol{\Lambda}^{(t+1)} \leftarrow \text{UPDATELAMBDA}(\{\text{P-rule}_i^{(t)}\}, \boldsymbol{\lambda}^{(t)}, \mathbf{m}^{(t-1)}, \boldsymbol{\Lambda}^{(t)})$ $\triangleright$ Alg. 2
 18: **until** maximum number of iterations is reached
 19: **return** **SELECTMODEL**(history)

4 Experimental Study

In order to evaluate whether AADA reduces manual fairness tuning while remaining competitive in accuracy–fairness trade-offs, we propose an experimental study on eight binary-classification datasets, using the Fair Fairness Benchmark (FFB) [14] as a standardised evaluation framework. We compare AADA against five representative in-processing fairness methods introduced in Section 2: ERM as the unconstrained reference, Prejudice Remover, HSIC, AdvDebias, and LAFTR.

Datasets. Table 1 summarises the datasets used in our study. Most of them are standard FFB benchmarks, with predefined preprocessing and declared sensitive attributes. We complement them with HIMS-Tunisia, a dataset introduced in this work: derived from the Tunisia Household International Migration Survey (MED-HIMS, 2020–2021) [17], it defines the task of predicting the legal entry status of a migrant from pre-departure features such as age, employment, and household characteristics. We consider gender, education level, and region of origin as sensitive attributes, all three are structurally correlated with the outcome, making HIMS-Tunisia a realistic multi-attribute test case for fairness methods.

Sensitive attribute identification. For FFB datasets, IDENTIFYSENSITIVE was queried on each dataset schema and sample prior to any experiment. In all cases, the orchestrator recovered the full set of attributes declared by the FFB benchmark. In several cases it returned additional candidates: for instance, on the Adult dataset it additionally identified **education** beyond the standard **sex**

Table 1. Summary of the eight datasets.

Dataset	Task	Sensitive attributes	Size	Notes
Adult [22]	Income > \$50K	sex, race	45,000	Standard FFB benchmark
German [16]	Credit risk	sex, age	1,000	Acc. ceiling 75–78%
COMPAS [1]	2-yr recidivism	race, sex	6,000	Acc. ceiling ~68%
Bank [26]	Deposit subscription	age	41,000	Strong class imbalance
KDD Census [8]	Income > \$50K	race, sex	292,000	~93% negative class
ACS [7]	Income > median	race, sex	100,000	Subsampled from 378K rows
UTKFace [36]	Age > median	ethnicity, gender	23,000	Image-based task
HIMS-Tunisia [17] (<i>new</i>)	Legal entry status	gender, education, region	3,161	First fairness study on this survey

and **race**. Rather than discarding these candidates automatically, we made a deliberate methodological choice to restrict experiments to the FFB attribute set, ensuring direct comparability with baseline methods. Prompt templates used for attribute identification are available in the public repository. On HIMS-Tunisia, where no prior attribute definition exists in the literature, the orchestrator identified three sensitive attributes (gender, education, and region) autonomously, with no manual intervention, demonstrating that AADA generalizes to unseen datasets where baseline methods require expert annotation.

Baselines. Baseline operating points on the seven FFB datasets are obtained with the official FFB implementations and protocol. On HIMS-Tunisia, which is absent from FFB, we ran the same FFB implementations ourselves under an identical protocol. On UTKFace, the comparison is restricted to Prejudice Remover and HSIC, as FFB does not provide adversarial methods (AdvDebias, LAFTR) for image data.

Implementation. Following the FFB protocol, the classifier architecture is dataset-type-dependent: a two-layer MLP with 256 units per layer and ReLU activations for tabular datasets, and ResNet-18 with ImageNet-pretrained weights for UTKFace. The adversary is intentionally kept small – a two-layer MLP with 32 units per layer and ReLU activations for all datasets, receiving only $\hat{y}_n \in (0, 1)$ – to reduce the risk that weak adversary performance is merely due to adversary underfitting [35]. Batch sizes follow the FFB protocol, for HIMS-Tunisia, absent from FFB, we use $b = 32$. Both agents are trained with Adam at $\eta_f = \eta_r = 10^{-3}$, with an 80/20 stratified train–test split and $N_{\text{pre}} = 10$ pre-training epochs. The adversarial loop runs for up to 25 iterations of $E = 50$ epochs each. Lambda update parameters are fixed across all datasets: $\eta_\lambda = 2.5$, $\beta = 0.7$, $\lambda_{\text{max}} = 20$, $\tau_{\text{fair}} = 80$, and $\lambda_i^{(0)} = 0$ unless warm-started from memory.

Since β averages the fairness signal over about $1/(1 - \beta)$ iterations [28], the standard default $\beta = 0.9$ [29, 20] over-smooths the non-stationary adversarial signal [32], so we use $\beta = 0.7$, which gave more stable λ trajectories and a better fairness–accuracy trade-off.

All experiments, baselines and AADA alike, are repeated over 10 random seeds. AADA is implemented in Python with PyTorch¹ and LangChain². Sensi-

¹ <https://pytorch.org>

² <https://www.langchain.com>

tive attribute identification uses Llama 3.1 served locally via Ollama³ to preserve confidentiality of dataset schemas and sample rows. Full details and reproducibility instructions are available at the public repository.⁴

Metrics. We report accuracy and P-rule (Eq. (10)), with $\tau_{\text{fair}} = 80$ corresponding to the EEOC four-fifths rule [10]. Our P-rule matches the `prule` metric of FFB [14], computed with the same decision threshold of 0.5, ensuring that baseline operating points and AADA are directly comparable.

4.1 Results

Figure 2 is an operating-point map. For each dataset–attribute pair, each baseline cloud reflects the accuracy–fairness trade-offs obtained by sweeping the fixed fairness penalty α ⁵ of that method, following the FFB protocol [14]. The exact role of α is method-dependent (an adversarial weight for AdvDebias and LAFTR, a mutual-information regularizer for Prejudice Remover, a kernel independence penalty for HSIC). The points of a given method that are not dominated by another point of the same method trace its empirical Pareto frontier, moving toward higher P-rule typically costs accuracy. AADA does not sweep α since it adapts $\lambda^{(t)}$ online and returns one operating point per attribute. All results, baselines and AADA alike, are averaged over 10 random seeds.

AADA (star marker) lands in or near the safe zone across most dataset–attribute pairs. On Bank, KDD, and ACS, baselines reach the safe zone only after sweeping α , AADA reaches a comparable point in a single run. On Adult, German, and HIMS-Tunisia, AADA covers all attributes simultaneously while each baseline cloud covers one attribute per run. On COMPAS, no method reaches P-rule ≥ 80 , a structural limitation unrelated to AADA.

Table 2 gives numerical confirmation: for each baseline we report the highest-accuracy point satisfying P-rule ≥ 80 across the full α sweep, giving each competitor its best possible result. On UTKFace, FFB provides no adversarial implementation for image data. We therefore implemented AdvDebias ourselves for this dataset, while LAFTR remains excluded. AADA reaches the safe zone on the same six datasets with accuracy within 1–2 points of the best baseline in every case, without a full penalty sweep.

4.2 Training Efficiency

A central motivation for AADA is to avoid the repeated training runs required by manual penalty sweep. We therefore analyze efficiency from two complementary angles: the per-epoch computational order of each method and the number of training runs induced by the α sweep. Table 3 compares the per-epoch cost of the considered methods.

³ <https://ollama.com>

⁴ https://github.com/wassim-fathallah/adversarial_learning2

⁵ Denoted λ in the FFB codebase, we write α to avoid confusion with the adaptive vector $\lambda^{(t)}$ of AADA.

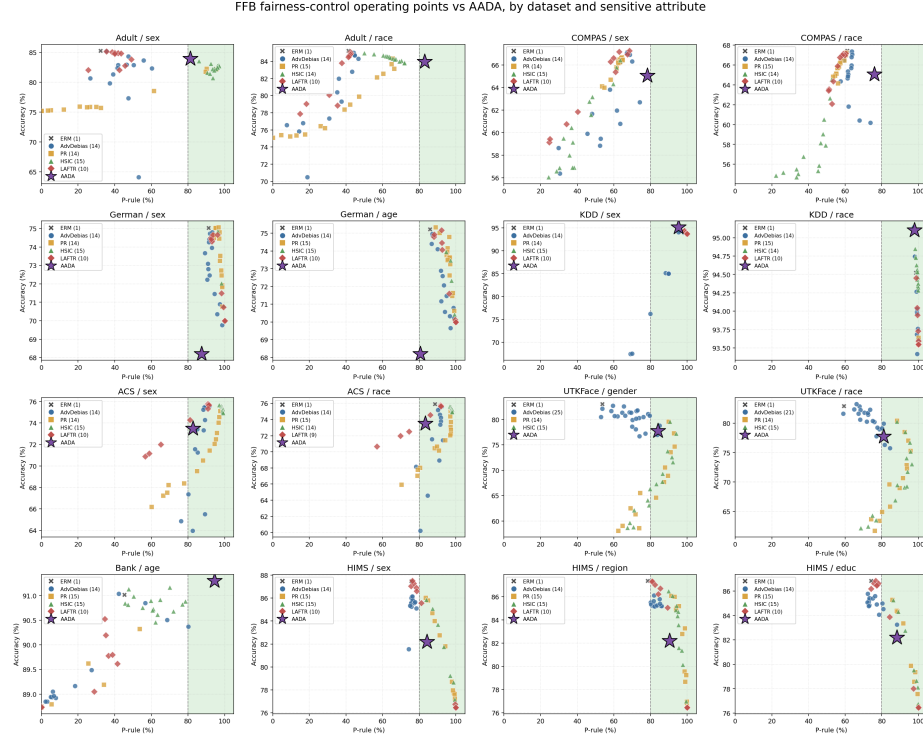


Fig. 2. Accuracy–P-rule operating points per dataset–attribute pair - Green region: safe zone ($P\text{-rule} \geq 80$) - Number in parentheses are the cardinality of the α sweep of each method.

Here, N denotes the number of training instances, d the input dimension, b the batch size, d_z the latent representation dimension, n_s the number of sensitive attributes, and C the cost of one forward–backward classifier update. Since the per-attribute lambda update is only $O(n_s)$ per iteration, AADA does not claim a lower per-epoch complexity. Its efficiency gain comes from reducing the number of full training runs.

Under the FFB protocol, each combination of α value, sensitive attribute, and random seed requires one independent training run. Taking the 14-value AdvDebias sweep as reference, this amounts to $14 \times n_s$ runs per dataset per seed, up to 42 on HIMS-Tunisia. AADA replaces this entire sweep with one adaptive run per seed. As reported in Table 4, the number of iterations used reflects dataset difficulty directly: KDD and Bank converge in one or two iterations because the fairness target is easily reachable, while COMPAS and UTKFace exhaust the full budget of 25 because the label distribution makes demographic parity structurally hard to satisfy.

Table 2. Accuracy (%) and P-rule (%) at the best trade-off operating point. P-rules in parentheses follow the attribute order. **Bold:** best P-rule in the row.

Dataset	Attr.	ERM	AdvDebias	LAFTR	HSIC	PR	AADA
		Acc / P-rule	Acc / P-rule	Acc / P-rule	Acc / P-rule	Acc / P-rule	Acc / (P-rules)
Adult	sex	85.3 / 32.2	82.3 / 60.2	83.9 / 49.0	83.6 / 85.9	82.3 / 90.1	<i>83.96</i> / (81.2, 82.9)
	race	85.3 / 41.2	84.3 / 46.8	84.9 / 42.6	83.9 / 71.9	83.2 / 66.3	
German	sex	75.0 / 91.1	74.8 / 93.1	74.7 / 93.5	74.9 / 96.3	75.4 / 95.5	68.20 / (87.3, 80.60)
	age	75.2 / 85.8	75.0 / 87.1	75.2 / 92.2	74.0 / 94.7	75.3 / 88.8	
COMPAS	race	67.5 / 61.1	67.1 / 63.8	67.2 / 60.3	65.9 / 55.7	67.3 / 60.6	<i>65.08</i> / (75.9 , 78.3)
	sex	67.1 / 67.4	67.1 / 67.1	67.3 / 68.5	66.4 / 63.4	66.5 / 64.7	
Bank	age	91.0 / 45.3	90.3 / 98.8	89.6 / 41.5	90.7 / 92.5	90.9 / 93.8	91.3 / 94.5
KDD	race	94.5 / 98.6	95.1 / 97.4	94.5 / 98.7	94.9 / 98.1	93.6 / 99.9	95.1 / (97.8, 95.3)
	sex	94.9 / 96.1	94.8 / 95.8	94.3 / 98.5	94.4 / 98.5	93.7 / 100	
ACS	race	77.7 / 86.7	76.7 / 92.8	75.8 / 91.6	79.2 / 98.1	78.1 / 98.0	73.5 / (83.2, 82.6)
	sex	73.2 / 89.4	76.9 / 86.4	75.8 / 90.8	79.0 / 98.5	77.2 / 99.2	
UTKFace	gender	83.1 / 53.6	78.8 / 84.9	—	80.2 / 89.7	79.4 / 89.5	<i>77.75</i> / (84, 81)
	race	82.9 / 59.4	75.7 / 84.4	—	80.8 / 88.4	80.5 / 87.0	
HIMS	gender	87.5 / 75.7	85.1 / 78.4	85.6 / 81.1	85.9 / 83.8	86.0 / 83.5	<i>82.2</i> / (84.1 , 90.4 , 88.3)
	educ	87.4 / 78.4	85.2 / 85.7	87.3 / 81.1	86.5 / 89.9	86.1 / 91.3	
		86.9 / 74.1	83.3 / 88.2	83.9 / 84.3	85.3 / 86.2	85.3 / 85.7	

*Note:*LAFTR is evaluated on the tabular datasets only and is not applied to the image dataset UTKFace.

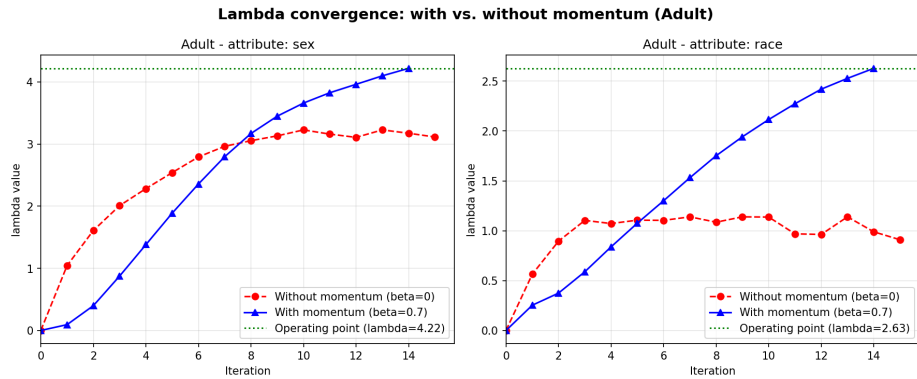
Table 3. Per-epoch computational cost.

Method	Per-epoch cost	Remark
ERM	$O(NC)$	standard SGD [11]
AdvDebias	$O(NC)$	adversary cost included in C [35]
Prejudice Remover	$O(Nd)$	regularized logistic regression [19]
HSIC	$O(NC + Nb)$	extra $b \times b$ Gram matrix per batch [13]
LAFTR	$O(Ndd_z)$	encoder-decoder bottleneck [24]
AADA	$O(NC)$	same training order, λ update is $O(n_s)$ per iteration

Effect of momentum. To assess the role of the momentum term in Eq. (12), we compare the evolution of λ_i with ($\beta = 0.7$) and without ($\beta = 0$) momentum. Figure 3 shows that momentum dampens oscillations and produces smoother lambda trajectories, supporting its use as a stabilizing component of the dynamic penalty update.

Table 4. Training budget of AADA vs. a 14-value α sweep per attribute (the FFB AdvDebias cardinality). Runs are counted per seed, both protocols use 10 seeds.

Dataset	n_s	Sweep runs	AADA runs	AADA iter. used
Adult	2	28	10	17
Bank	1	14	10	1
COMPAS	2	28	10	25
German	2	28	10	14.6
KDD	2	28	10	1
ACS	2	28	10	9
UTKFace	2	28	3	19.3
HIMS	3	42	3	14

**Fig. 3.** λ trajectories on Adult with ($\beta=0.7$, blue) and without ($\beta=0$, red) momentum. Dotted green line: reference operating point selected by the best-performing run.

5 Conclusion

This paper proposed AADA, an agentic framework for adversarial debiasing in which an orchestrator identifies sensitive attributes and adapts the fairness penalty $\lambda^{(t)}$ during training. Experiments on the benchmark datasets and HIMS-Tunisia show that AADA reaches or approaches the target P-rule while reducing the need for the repeated penalty sweeps typically required by fixed-penalty adversarial debiasing.

HIMS-Tunisia provides the clearest test of this contribution. In a setting with no prior fairness configuration, the orchestrator identified gender, education, and region as sensitive attributes and guided the debiasing process without per-attribute manual setup. The same orchestrator also records which attributes were considered and how the penalty evolved during training, the low-code layer uses this information to generate fairness reports, positioning AADA not only as a training method but also as a step toward auditable fairness pipelines.

The main limitation is the dependence of sensitive-attribute identification on the locally served LLM. Future work should evaluate this step across different LLMs, measure the gain from memory-based λ_i^{init} warm-starts, and test whether the same orchestrator-driven identification extends to text and multimodal data.

References

1. Angwin, J., Larson, J., Mattu, S., Kirchner, L.: Machine bias: There’s software used across the country to predict future criminals. and it’s biased against blacks. ProPublica (2016)
2. Barocas, S., Hardt, M., Narayanan, A.: Fairness and Machine Learning. fairml-book.org (2019), <http://www.fairmlbook.org>
3. Bolukbasi, T., Chang, K.W., Zou, J., Saligrama, V., Kalai, A.: Man is to computer programmer as woman is to homemaker? Debiasing word embeddings. In: NeurIPS (2016)
4. Buolamwini, J., Gebru, T.: Gender shades: Intersectional accuracy disparities in commercial gender classification. In: FAccT (2018)
5. Chouldechova, A.: Fair prediction with disparate impact. *Big Data* **5**(2), 153–163 (2017)
6. Dastin, J.: Amazon scraps secret AI recruiting tool that showed bias against women. Reuters (2018)
7. Ding, F., Hardt, M., Miller, J., Schmidt, L.: Retiring adult: New datasets for fair machine learning. In: NeurIPS (2021)
8. Dua, D., Graff, C.: UCI machine learning repository (2019), university of California, Irvine
9. Dwork, C., Hardt, M., Pitassi, T., Reingold, O., Zemel, R.: Fairness through awareness. In: ITCS (2012)
10. Feldman, M., Friedler, S., Moeller, J., Scheidegger, C., Venkatasubramanian, S.: Certifying and removing disparate impact. In: KDD (2015)
11. Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. MIT Press (2016), <http://www.deeplearningbook.org>
12. Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., Bengio, Y.: Generative adversarial nets. In: NeurIPS (2014)
13. Gretton, A., Bousquet, O., Smola, A.J., Schölkopf, B.: Measuring statistical dependence with Hilbert-Schmidt norms. In: Proceedings of the 16th International Conference on Algorithmic Learning Theory (ALT). pp. 63–77. Springer (2005)
14. Han, X., et al.: FFB: A fair fairness benchmark for in-processing group fairness methods. In: ICLR (2024)
15. Hardt, M., Price, E., Srebro, N.: Equality of opportunity in supervised learning. In: NeurIPS (2016)
16. Hofmann, H.: Statlog (German credit data) (1994), UCI Machine Learning Repository
17. Institut National de la Statistique: Tunisia household international migration survey (HIMS-Tunisia). Tech. rep., MED-HIMS Programme, Tunisia (2020)
18. Kamiran, F., Calders, T.: Data preprocessing techniques for classification without discrimination. *Knowledge and Information Systems* **33**(1), 1–33 (2012)
19. Kamishima, T., Akaho, S., Asoh, H.: Fairness-aware classifier with prejudice remover regularizer. In: ECML PKDD (2012)

20. Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. In: 3rd International Conference on Learning Representations (ICLR) (2015)
21. Kleinberg, J., Mullainathan, S., Raghavan, M.: Inherent trade-offs in the fair determination of risk scores. In: ITCS (2017)
22. Kohavi, R.: Scaling up the accuracy of Naive-Bayes classifiers: A decision-tree hybrid (1996), adult dataset, UCI Machine Learning Repository
23. Li, Y., Chang, S., Chen, P.Y., Das, P., Ramamurthy, K.N.: Fair representations with guarantees for multiple groups via kernel independence criterion. In: AAAI (2019)
24. Madras, D., Creager, E., Pitassi, T., Zemel, R.: Learning adversarially fair and transferable representations. In: ICML (2018)
25. Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K., Galstyan, A.: A survey on bias and fairness in machine learning. *ACM Computing Surveys* **54**(6), 1–35 (2021)
26. Moro, S., Cortez, P., Rita, P.: A data-driven approach to predict the success of bank telemarketing. *Decision Support Systems* **62**, 22–31 (2014)
27. Ntoutsi, E., et al.: Bias in data-driven AI systems — an introductory survey. *WIREs Data Mining and Knowledge Discovery* **10**(3) (2020)
28. Qian, N.: On the momentum term in gradient descent learning algorithms. *Neural Networks* **12**(1), 145–151 (1999)
29. Sutskever, I., Martens, J., Dahl, G., Hinton, G.: On the importance of initialization and momentum in deep learning. In: Proceedings of the 30th International Conference on Machine Learning (ICML). vol. 28, pp. 1139–1147. PMLR (2013)
30. Verma, S., Rubin, J.: Fairness definitions explained. In: FairWare (2018)
31. Wooldridge, M., Jennings, N.R.: Intelligent agents: Theory and practice. *The Knowledge Engineering Review* **10**(2) (1995)
32. Yazıcı, Y., Foo, C.S., Winkler, S., Yap, K.H., Piliouras, G., Chandrasekhar, V.: The unusual effectiveness of averaging in GAN training. In: 7th International Conference on Learning Representations (ICLR) (2019)
33. Zafar, M.B., Valera, I., Rodriguez, M., Gummadi, K.: Fairness constraints: Mechanisms for fair classification. In: AISTATS (2017)
34. Zemel, R., Wu, Y., Swersky, K., Pitassi, T., Dwork, C.: Learning fair representations. In: ICML (2013)
35. Zhang, B.H., Lemoine, B., Mitchell, M.: Mitigating unwanted biases with adversarial learning. In: AIES (2018)
36. Zhang, Z., Song, Y., Qi, H.: Age progression/regression by conditional adversarial autoencoder. In: CVPR (2017)